lexsync: A cross-platform pipeline for multidimensional lexical optimisation and hardware-timed experiment generation

Pablo Bernabeu

2026-06-13

Psycholinguistic stimulus preparation is fragmented across tools that match words, tools that present them and the separate R and Python ecosystems in which those tools live. We present 'lexsync', a research-grade toolkit, released as a 'CRAN'-ready R package and a structurally identical 'PyPI'-ready Python package, that unifies the whole path from corpus to laboratory. From a word-frequency corpus in any of dozens of languages 'lexsync' selects stimuli that are matched in parallel across several lexical dimensions (length, frequency, orthographic neighbourhood density and OLD20), counterbalances them, reports the realised match quality and then generates ready-to-run 'PsychoPy', 'OpenSesame' and browser-based 'jsPsych' experiments, the two laboratory targets time-locking their electroencephalography triggers to stimulus onset. The trial is described declaratively as a sequence of events, so the same engine builds factorial word studies, lexical decision with generated pseudowords, priming and self-paced reading from configuration rather than code. The matching and generation are deterministic, so the two engines select identical materials; we demonstrate this on eight worked examples in English, Spanish and Mandarin Chinese — across word, pseudoword, paired and sentence stimuli — one of which shows that the workflow extends to a logographic writing system. The generation step requires no presentation software and no hardware, which keeps the whole workflow reproducible. 'lexsync' generalises the FAIR artificial-grammar workflow of an existing longitudinal study and is distributed under open licences with a machine-readable corpus registry.

1 Introduction

The credibility of psycholinguistic and neurolinguistic research rests on carefully controlled stimuli and on precise timing. Both are laborious to achieve, and the tools that support them are scattered. A researcher who wishes to contrast, say, high- and low-frequency words while equating length and orthographic neighbourhood typically matches the items in one program, lays out the trial structure in another and writes the trigger code that aligns each word with the electroencephalography (EEG) recording by hand. Each hand-off is an opportunity for error and a barrier to reuse. The 'FAIR' guiding principles (Wilkinson et al., 2016) — that research objects be findable, accessible, interoperable and reusable — apply as much to this preparatory machinery as to the data it ultimately yields, yet stimulus pipelines are seldom shared in a form that others can run end to end.

This preparatory machinery is where a quiet reproducibility problem lives. Surveys of open-science practice in linguistics find that experimental materials are rarely shared: across a broad sample only about one article in six stated that its materials were available, and the sharing of materials, data and protocols has scarcely improved over a decade (Bochynska et al., 2023). Yet materials are a precondition both for replicating a result and for judging how far it generalises. The gap compounds an analytic one: quantitative phonetic and psycholinguistic work affords many undocumented choices — how a stimulus set is assembled, how a measure is quantified — and exploiting this flexibility inflates false positives (Coretta et al., 2023; Roettger, 2019; Roettger et al., 2019). Transparent, runnable, shareable materials are part of the remedy these authors call for.

There is also a statistical reason to take materials seriously. The items in an experiment are a sample from a larger population of possible items, and a conclusion is meant to hold for that population, not merely for the words actually shown. Treating items as a fixed rather than a random factor inflates Type I error — the language-as-fixed-effect fallacy (Clark, 1973) — and recent work recasts the same point as a *generalizability crisis*: researchers routinely model participants as a random factor but not stimuli, and so claim more than their designs license (Yarkoni, 2020). A tool that makes the construction of a matched, balanced item set explicit, reproducible and shareable is therefore not a convenience but part of the methodological infrastructure that careful inference requires.

Two families of tool address parts of the problem. The first matches or equates word stimuli on lexical properties. 'LexOPS' (Taylor et al., 2020) generates controlled word lists for arbitrary factorial designs, with both per-dimension tolerances and weighted multivariate Euclidean matching; 'LIBRA' (Lintz et al., 2021) equates lists across conditions using a resampling algorithm; and 'LASTU' (Itkonen et al., 2024) provides searchable access to a language-specific lexicon. These tools are mature and widely used, but they end at the stimulus list: none generates the experiment that presents it, and most are tied to a single language and to one programming ecosystem. The second family builds and runs experiments. 'PsychoPy' (Peirce et al., 2019) and 'OpenSesame' (Mathôt et al., 2012) offer precise spatial and temporal control over stimulus presentation and support the hardware triggers that ERP research requires, but they take the stimulus set as given and do nothing to construct or to balance it.

The gap, then, is not a missing capability but a missing integration. No single, installable framework spans many-language corpus access, parallel multidimensional matching, counterbalancing and the automated generation of hardware-timed presentation scripts, and none does so across both the R and the Python ecosystems and both major open presentation platforms. A second, subtler gap concerns timing. In the artificial-grammar workflow that motivated this work (González Alonso et al., 2025), EEG markers were sent from an experiment item that followed the word, so that only the first marker was even approximately aligned with stimulus onset; the others were sequence-ordered. This is a common pattern and an avoidable source of jitter.

We present 'lexsync', a toolkit that fills these gaps; both presentation targets time-lock the marker to stimulus onset (see below). It is released as two independent but structurally identical packages — a 'CRAN'-ready R package and a 'PyPI'-ready Python package — that share module names, function names and behaviour. From a corpus 'lexsync' selects matched stimuli, counterbalances and documents them, then writes a 'PsychoPy' script in which each onset trigger is bound to the exact screen flip on which the critical stimulus appears, using `win.callOnFlip`, together with a complete 'OpenSesame' experiment. Crucially the matching is deterministic, so the R and Python engines select identical materials. The trial itself is not hard-wired to a single task: a design declares an ordered sequence of trial *events*, and both backends render that sequence, so the same machinery produces a factorial word study, a lexical-decision task

with generated pseudowords, a primed design or a self-paced reading study from configuration rather than new code. 'lexsync' thus generalises the materials workflow of González Alonso et al. (2025) — itself an artificial-grammar study — into a reusable, FAIR-compliant tool for many psycholinguistic paradigms.

2 The lexsync packages

2.1 Architecture and cross-language design

Each package is organised into the same modules: `querying` (load a lexicon or an item table and compute lexical dimensions), `matching` (the constraint engine), `generation` (deterministic pseudoword generation), `paradigms` (the trial-event registry), `counterbalancing`, `validation` (match-quality reporting), `scripting` (experiment generation), `datasheet` (the materials datasheet and pre-registration template), `logging` (the automated run log) and `corpora` (the registry). A single orchestrator, `run_pipeline`, drives them in an explicit loop over designs and languages, so that adding a language, a design or a paradigm is a matter of supplying a configuration file (and, where relevant, a lexicon or item table) rather than writing code. The two packages are deliberately lean: the R package depends only on 'readr', 'yaml', 'stringdist' and 'jsonlite' beyond base R, and the Python package on 'pandas', 'numpy', 'PyYAML', 'rapidfuzz' and 'scipy'.

2.2 A many-language corpus registry

Stimuli are only as good as the corpus behind them. 'lexsync' reads a machine-readable registry that records, for each supported corpus, its language, access URL, licence and citation. Two connectors are provided. The first exposes the curated SUBTLEX family and the 'openlexicon' collection — for example SUBTLEX-UK for British English (Heuven et al., 2014), SUBTLEX-ESP for Spanish (Cuetos et al., 2011) and SUBTLEX-US (Brysbaert & New, 2009) — each individually citable and redistributable under CC BY-SA terms. The second is the 'wordfreq' package (Speer, 2022), which yields frequencies for roughly forty languages through a single dependency. The demonstrations below build their English, Spanish and Mandarin Chinese lexica with 'wordfreq', which aggregates subtitle and web corpora including the SUBTLEX lists; the registry documents the curated corpora — among them SUBTLEX-CH for Chinese (Cai & Brysbaert, 2010) — as alternatives. Small example lexica are bundled inside each package for offline use and testing, while full corpora are fetched on demand into a user cache, keeping the installed packages within distribution size limits.

2.3 The multidimensional matching algorithm

For every word 'lexsync' records its length, its frequency and two orthographic-neighbourhood measures: Coltheart's N, the number of words of the same length differing by a single letter (Coltheart et al., 1977), and OLD20, the mean Levenshtein distance to the twenty nearest neighbours (Yarkoni et al., 2008). The neighbourhood measures are computed transparently from the lexicon rather than taken on trust.

Matching proceeds in two deterministic stages. First, a tolerance pre-filter keeps only candidates that fall within $M \pm k \cdot SD$ of the anchor condition on each control dimension, generalising the windowed selection of the original workflow. Second, every remaining condition is matched to the anchor item by item: each dimension is standardised, the standardised Euclidean distance

from each candidate to the target item is computed and the nearest unused candidate is assigned, with ties broken by a stable, locale-independent key. Because no random number generator is used and string comparisons are performed in byte order in both engines, the R and Python packages select an identical matched selection from identical input — a parity we verify below.

The per-anchor procedure fixes one condition and matches the others to it, which is ideal when the manipulated dimension is statistically independent of the controls. When the manipulation is intrinsically correlated with a control dimension — as orthographic neighbourhood density is with word length and frequency — fixing one condition first can leave a residual difference on the control, because the anchor items need not have close counterparts in the other condition. For such designs 'lexsync' offers a `joint` matching method, selected with a single line in the design configuration. Rather than privileging one condition, it scores every cross-condition pair on the standardised control dimensions and greedily retains the n cheapest disjoint pairs, so only items that have a close counterpart in the opposite condition are kept. The two conditions are thereby equated on the controls even when the manipulation is confounded with them, at the cost of drawing from a slightly larger candidate pool. The method is as deterministic and cross-engine identical as the per-anchor one (costs are rounded before the byte-order tie-break), and it is the method used for the neighbourhood-density demonstrations below.

2.4 Counterbalancing and reporting

Matched items are assigned to presentation lists and a reproducible trial order, and a participant table crosses any counterbalancing factors. 'lexsync' then writes a match-quality report and an automated, comprehensive run log recording each stage, its parameters, the seed, package and corpus provenance and a fingerprint of every file written.

The report is deliberately built around effect sizes rather than significance tests. For each controlled dimension it gives the per-condition descriptive statistics and the standardised mean difference (Cohen's d) between conditions, together with a 90% confidence interval on that difference. This ordering is a considered response to a common error in nuisance control: treating a non-significant difference test as proof that a variable is matched (Sassenhagen & Alday, 2016). A non-significant test is not evidence of equivalence, and whether a real difference reaches significance depends on the number of items, so the conclusion "matched" can be manufactured by using few items. The standardised mean difference is, by contrast, a direct measure of the realised imbalance, and its confidence interval makes the sampling uncertainty — and hence the dependence on the number of items — explicit: with few items the interval is wide, so a small point estimate cannot be over-read, and the upper limit of the interval on $|d|$ is the largest imbalance still consistent with the chosen stimuli. The report also includes a two one-sided tests (TOST) equivalence test (Lakens, 2017) as a complementary, bound-referenced verdict; the 90% interval is exactly the interval whose lying inside the bound corresponds to a TOST at the .05 level, so the two are coherent and the interval is the more transparent of the pair. Together they make the realised control explicit, in the spirit of the original workflow's balance checks, and the run log makes a run auditable.

Finally, each design is accompanied by a *materials datasheet* — a machine-readable JSON record and a human-readable Markdown summary of its provenance: the corpus or item table and its checksum, the dimensions and how they were computed, the selection or matching method and its parameters, the realised control (the effect sizes, intervals and equivalence verdicts above), the counterbalancing recipe, and the seed, software versions and file checksums needed to reproduce the materials exactly. The datasheet also emits an auto-filled Methods paragraph and a pre-registration template with its Materials section completed. This is a direct response to the finding that experimental materials are rarely shared in a usable form (Bochynska et

al., 2023; Roettger, 2019): the materials, the procedure and the record of how they were made travel together.

2.5 Hardware time-locking

The 'scripting' module emits two presentation targets from the same matched stimuli. The 'PsychoPy' script binds each onset trigger to the stimulus flip:

```
win.callOnFlip(port.setData, target_trigger)   # written on the onset flip
for frame in range(WORD_DURATION_FRAMES):
    word_stim.draw()
    win.flip()
    if frame == RESET_AFTER_FRAMES:
        win.callOnFlip(port.setData, 0)
```

A mock port prints the codes when no parallel-port driver is present, so the script runs end to end with no hardware. The 'OpenSesame' export is a complete, plain-text `.osexp`, generated programmatically so that its tab-sensitive format is always valid and modelled on a proven-working experiment from González Alonso et al. (2025). In it the word is drawn and shown from an inline script, and the marker is sent immediately after `show()` returns, supporting both a parallel-port and a serial backend with a test-mode fallback. Because `show()` blocks until the display refresh, the marker is written right after the verified flip, so both targets are time-locked to stimulus onset: 'PsychoPy' binds the marker to the flip with `win.callOnFlip`, and 'OpenSesame' writes it immediately after the refresh-blocking `show()`, the platform's recommended method. We recommend a photodiode check in either case. Notably, neither the matching nor the script generation imports 'PsychoPy' or any serial library: generation only writes text, and the presentation software is needed solely at run time.

2.6 Paradigm generality: the trial-event model

The original workflow built one kind of trial — a fixation cross, a single word and a response — and wrote it into the experiment by hand. 'lexsync' replaces that fixed structure with a declarative one. A trial is an ordered list of *events*, each a small record naming its type (a fixation, a piece of text, a mask, a blank interval, a region-by-region passage, a response or a comprehension question), its content (a literal such as a fixation cross, or a reference to a field of the trial such as the target word or the sentence), its duration, and an optional EEG trigger that may be locked to the event's onset. A *paradigm* is simply a named default sequence of such events together with the trial fields it requires; a design either selects a paradigm or supplies its own event list. All three presentation backends iterate the same list — the 'PsychoPy' script interprets it at run time, the 'OpenSesame' generator emits one block per event, and the 'jsPsych' generator builds a browser timeline from it — so adding a paradigm is a matter of configuration, not backend code, and the targets stay in step by construction. The 'jsPsych' target is a single self-contained HTML file that runs in any modern browser, letting anyone reproduce the exact procedure online from the same materials; because a browser cannot drive a parallel port, each onset marker is recorded in the trial's data rather than written to hardware (online EEG synchronisation would route it through WebSerial or a photodiode). Stimulus text is always written as data into the conditions file the experiment reads, never interpolated into the generated code, and every value is validated (no control characters; bounded length) before it is written, so a malformed item cannot corrupt a script.

Items reach a design from one of three sources. The corpus source is the matching engine described above. The *table* source loads a user-supplied set of items — prime–target pairs, or sentences segmented into regions with a comprehension question — validated against the paradigm's required fields and counterbalanced by a Latin square that shows each item once per list in a rotated condition, so a target is never repeated within a list. The *generate* source builds pseudowords. For each real word 'lexsync' derives a length-matched, orthographically legal non-word by changing the fewest letters such that every resulting letter bigram is attested in the corpus and the form is not itself a word, choosing among the legal candidates the most bigram-plausible with a byte-order tie-break. This is a deterministic, cross-engine-reproducible relative of 'Wuggy' (Keuleers & Brysbaert, 2010): where Wuggy models subsyllabic structure, 'lexsync' trades that for exact length matching and identical output from the R and Python engines.

2.7 Reproducibility and FAIR compliance

'lexsync' declares its dependencies in a `DESCRIPTION` file for R and pinned requirements for Python, sets seeds from a single configuration file, and carries a 'CITATION.cff', an MIT licence for the code and CC BY-SA terms for the bundled corpus derivatives with full attribution. The R package passes `R CMD check --as-cran`; the Python package builds a wheel and a source distribution that pass `twine check`. Both ship unit test suites, including a mock-engine test that confirms the generated 'PsychoPy' script writes its onset trigger on the word's first flip and a structural validator for the generated `.osexp`.

3 Worked examples

We illustrate 'lexsync' with eight demonstrations. The first five are corpus-matched word contrasts; the last three exercise other paradigms. Of the matched contrasts, four are alphabetic: a high- versus low-frequency contrast and a dense- versus sparse-neighbourhood contrast, each in English and in Spanish, drawn from thirty-thousand-word lexica. The frequency contrast matches items on length, neighbourhood density and OLD20; the neighbourhood contrast matches on length and frequency. The fifth is a high- versus low-frequency contrast in Mandarin Chinese, included to show that the pipeline generalises to a logographic writing system (see *Generalisation to a logographic script*, below). The remaining three — lexical decision, priming and self-paced reading — show the trial-event model carrying word, pseudoword, paired and sentence stimuli (see *Beyond word lists*, below). Each demonstration was produced by both the R and the Python engine. All tables and figures in this section are computed at render time from the generated output. The tables that follow summarise the five matched word contrasts, for which a controlled-versus-manipulated report is defined.

Table 1: *Manipulated and Controlled Standardised Mean Differences by Demonstration. Note.* Each demonstration, its manipulated dimension and the absolute standardised mean difference (Cohen's d) of the manipulated dimension and of the most-different controlled dimension. The manipulation is large in every case; the controlled dimensions remain small.

Demonstration	Manipulated dimension	|d| manipulated	largest |d| controlled
English: frequency contrast	Frequency (Zipf)	5.27	0.04

Table 1: *Manipulated and Controlled Standardised Mean Differences by Demonstration. Note.* Each demonstration, its manipulated dimension and the absolute standardised mean difference (Cohen's *d*) of the manipulated dimension and of the most-different controlled dimension. The manipulation is large in every case; the controlled dimensions remain small.

Demonstration	Manipulated dimension	|d| manipulated	largest |d| controlled
English: neighbourhood contrast	Neighbourhood N	3.11	0.00
Spanish: frequency contrast	Frequency (Zipf)	5.55	0.08
Spanish: neighbourhood contrast	OLD20	2.42	0.00
Chinese: frequency contrast	Frequency (Zipf)	6.00	0.11

Table 1 shows that the manipulated dimension is strongly separated in every demonstration. The control dimensions, by contrast, show standardised mean differences well below the 0.5-standard-deviation bound. Figure 1 plots the absolute standardised mean difference of each dimension against the anchor; the manipulated dimension stands far to the right of the 0.5-standard-deviation reference bound (dashed line) while the controlled dimensions cluster near zero.

The realised control is best read from the effect sizes and their intervals. Table 2 lists, for the controlled dimensions of each demonstration, Cohen's *d* against the anchor, its 90% confidence interval and the complementary TOST verdict at the 0.5-*SD* bound.

Table 2: *Realised Control on the Controlled Dimensions: Effect Sizes With Confidence Intervals and the Complementary Equivalence Test. Note.* For each demonstration's controlled (matched) dimensions: the standardised mean difference (Cohen's *d*) against the anchor, its 90% confidence interval, and the two one-sided tests (TOST) *p* value and verdict at the 0.5-*SD* bound. The 90% interval is the one coherent with a .05-level TOST. R engine.

Demonstration	Dimension	Cohen's d	90% CI	TOST p	Equivalent
Chinese: frequency contrast	Length	0.000	[0.00, 0.00]	0.000	TRUE
Chinese: frequency contrast	Neighbourhood N	0.071	[-0.19, 0.33]	0.004	TRUE
Chinese: frequency contrast	OLD20	0.106	[-0.15, 0.37]	0.007	TRUE
English: frequency contrast	Length	0.027	[-0.23, 0.29]	0.002	TRUE
English: frequency contrast	Neighbourhood N	0.038	[-0.22, 0.30]	0.002	TRUE
English: frequency contrast	OLD20	0.009	[-0.25, 0.27]	0.001	TRUE
English: neighbourhood contrast	Length	0.000	[-0.26, 0.26]	0.001	TRUE
English: neighbourhood contrast	Frequency (Zipf)	0.000	[-0.26, 0.26]	0.001	TRUE
Spanish: frequency contrast	Length	0.049	[-0.21, 0.31]	0.002	TRUE

Table 2: *Realised Control on the Controlled Dimensions: Effect Sizes With Confidence Intervals and the Complementary Equivalence Test. Note.* For each demonstration's controlled (matched) dimensions: the standardised mean difference (Cohen's d) against the anchor, its 90% confidence interval, and the two one-sided tests (TOST) p value and verdict at the 0.5-*SD* bound. The 90% interval is the one coherent with a .05-level TOST. R engine.

Demonstration	Dimension	Cohen's d	90% CI	TOST p	Equivalent
Spanish: frequency contrast	Neighbourhood N	0.077	[-0.18, 0.34]	0.004	TRUE
Spanish: frequency contrast	OLD20	0.006	[-0.26, 0.27]	0.001	TRUE
Spanish: neighbourhood contrast	Length	0.000	[-0.26, 0.26]	0.001	TRUE
Spanish: neighbourhood contrast	Frequency (Zipf)	0.000	[-0.26, 0.26]	0.001	TRUE

In every demonstration each controlled dimension's 90% confidence interval lies within the 0.5-*SD* bound ($n = 80$ per condition), so the realised imbalance is small even at the interval's far end — not merely non-significant. The TOST agrees, declaring each dimension equivalent at the .05 level, but the interval is the more informative statement because it would widen, and the claim would weaken, were the item count reduced. The neighbourhood contrasts are the demanding case: neighbourhood density is intrinsically correlated with word length and frequency, so a per-anchor match leaves a residual on length (Cohen's d 0.25). The `joint` matching method removes it: by selecting the best-matched dense–sparse pairs jointly it equates length and frequency almost exactly (d 0.00 on both, with intervals tightly around zero) while the density contrast remains an order of magnitude larger ($d > 2$). The confound is thus a solved engineering problem rather than an inherent limit on the design, and 'lexsync' reports the realised control transparently — effect size and interval first, equivalence test as a bound-referenced complement (Lakens, 2017; Sassenhagen & Alday, 2016).

The per-condition descriptive statistics make the control concrete. In the English frequency contrast (Table 3) frequency is separated by roughly one and a half Zipf units while length, neighbourhood density and OLD20 are almost identical across conditions. In the English neighbourhood contrast (Table 4) neighbourhood density is separated by an order of magnitude with length and frequency held constant; OLD20, which is mathematically related to N, moves with the manipulation rather than being held constant, as expected.

Table 3: *Descriptive Statistics per Condition: English Frequency Contrast. Note.* R engine.

Condition	Dimension	n	Mean	SD	Min	Median	Max
high frequency	Length	80	4.700	1.427	3.00	4.000	8.00
high frequency	Frequency (Zipf)	80	5.578	0.369	5.20	5.455	6.98
high frequency	Neighbourhood N	80	7.812	8.344	0.00	5.000	34.00
high frequency	OLD20	80	1.629	0.541	1.00	1.600	3.10
low frequency	Length	80	4.662	1.350	3.00	4.000	7.00
low frequency	Frequency (Zipf)	80	4.064	0.170	3.82	4.035	4.40
low frequency	Neighbourhood N	80	7.513	7.480	0.00	5.000	24.00
low frequency	OLD20	80	1.624	0.518	1.00	1.600	2.70

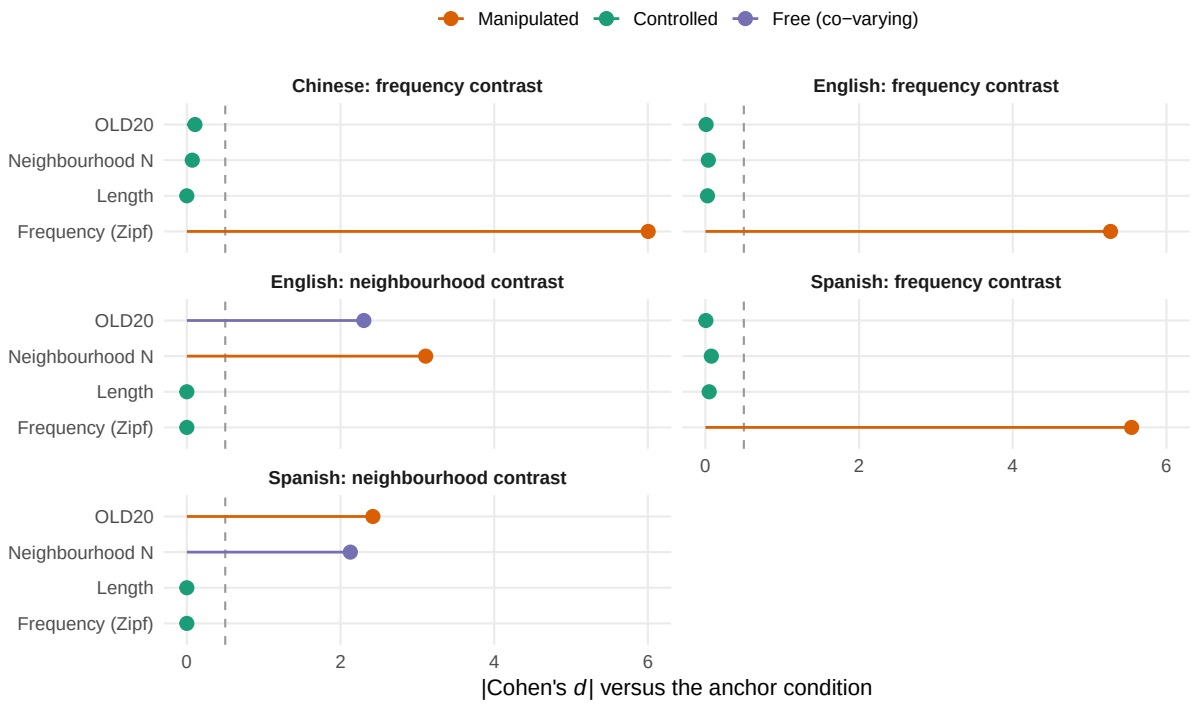

Figure 1: *Absolute Standardised Mean Differences by Dimension and Demonstration. Note.* Absolute standardised mean difference (Cohen's d) of each dimension against the anchor condition. The dashed line is the 0.5-*SD* reference bound (the smallest effect of interest for the equivalence test). Manipulated dimensions (orange) lie far to the right; matched dimensions (green) cluster near zero. In the neighbourhood contrasts OLD20 is free to co-vary (purple), reflecting its known correlation with Coltheart's N.

Table 4: *Descriptive Statistics per Condition: English Neighbourhood-Density Contrast. Note.* R engine.

Condition	Dimension	n	Mean	SD	Min	Median	Max
dense neighbourhood	Length	80	4.787	0.650	4.00	5.000	6.00
dense neighbourhood	Frequency (Zipf)	80	4.249	0.311	3.81	4.225	5.15
dense neighbourhood	Neighbourhood N	80	8.363	3.530	5.00	7.500	20.00
dense neighbourhood	OLD20	80	1.448	0.197	1.00	1.500	1.75
sparse neighbourhood	Length	80	4.787	0.650	4.00	5.000	6.00
sparse neighbourhood	Frequency (Zipf)	80	4.249	0.311	3.81	4.225	5.15
sparse neighbourhood	Neighbourhood N	80	0.525	0.503	0.00	1.000	1.00
sparse neighbourhood	OLD20	80	1.970	0.253	1.60	1.900	2.95

3.1 Generalisation to a logographic script

The orthographic-neighbourhood measures operate on character strings, so the pipeline is not tied to alphabetic writing. To show this concretely, the fifth demonstration is a high- versus low-frequency contrast in Mandarin Chinese. Its lexicon is derived from the same engine as the others — `wordfreq`'s Chinese data (Speer, 2022) — and the two conditions are two-character words, the dominant word form in Chinese. Here 'length' is the number of characters, and Coltheart's N and OLD20 are computed over characters rather than letters; we refer to them as character-level neighbourhood measures. Adding the demonstration required no change to the matching, counterbalancing or scripting code: a design configuration names the Chinese lexicon and a glyph-complete font, and the existing machinery does the rest.

Table 5 shows the result. Frequency is separated by about one and a half Zipf units (Cohen's d 6), while character-length is held constant at two and the character-level neighbourhood measures are equated across conditions (each with a 90% confidence interval on d lying within the 0.5-*SD* bound, and correspondingly TOST-equivalent). Chinese two-character words have dense orthographic neighbourhoods — tens of single-character substitutions each, an order of magnitude more than the alphabetic words above — and 'lexsync' equates this dense, skewed distribution as readily as the sparse alphabetic one. The generated 'PsychoPy' and 'OpenSesame' experiments set a CJK font, so the characters render directly; the selection is byte-identical across the R and Python engines, exactly as for the alphabetic demonstrations.

Table 5: *Descriptive Statistics per Condition: Mandarin Chinese Frequency Contrast. Note.* Length is in characters; neighbourhood N and OLD20 are character-level measures. R engine.

Condition	Dimension	n	Mean	SD	Min	Median	Max
high frequency	Length	80	2.000	0.000	2.0	2.000	2.00
high frequency	Frequency (Zipf)	80	5.324	0.294	5.0	5.245	6.56
high frequency	Neighbourhood N	80	69.825	64.807	8.0	51.500	307.00
high frequency	OLD20	80	1.038	0.111	1.0	1.000	1.55
low frequency	Length	80	2.000	0.000	2.0	2.000	2.00
low frequency	Frequency (Zipf)	80	3.810	0.201	3.5	3.810	4.20
low frequency	Neighbourhood N	80	65.662	52.159	14.0	51.500	199.00
low frequency	OLD20	80	1.028	0.074	1.0	1.000	1.25

Finally, because the matcher is deterministic the two engines select the same matched (word, condition) pairs rather than merely similar ones (the seeded trial order differs, as the R and Python random-number generators do). Across the five matched word contrasts all 800 of the selected (word, condition) pairs were identical between the R and the Python engine, and every statistic they report agrees to the precision shown. Table 6 makes this concrete for the English frequency contrast: the standardised mean difference of each dimension, computed independently in the two languages, coincides to the precision shown.

Table 6: *Cross-Engine Parity for the English Frequency Contrast. Note.* Each dimension's standardised mean difference (Cohen's d) against the anchor, computed independently by the R and Python engines. The columns are identical to three decimal places.

Dimension	Cohen's d (R engine)	Cohen's d (Python engine)
Length	0.027	0.027
Frequency (Zipf)	5.272	5.272
Neighbourhood N	0.038	0.038
OLD20	0.009	0.009

3.2 Beyond word lists: three paradigms

The same engine builds tasks that are not word lists. Three further demonstrations, each in English, exercise a different stimulus structure and trial procedure through the trial-event model. A **lexical-decision** design contrasts 60 real words with 60 pseudowords generated to match them in length; the generated non-words are orthographically legal and absent from the corpus, and length is equated by construction (Cohen's $d = 0.00$). A **priming** design draws 12 prime–target pairs from an item table and contrasts related with unrelated primes, its trial inserting a brief prime and a mask before the onset-locked target. A **self-paced reading** design presents 10 sentences region by region, the grammatical–ungrammatical manipulation falling on the onset-locked critical region and followed by a comprehension question. Each design compiles from the same event list to a runnable 'PsychoPy' script, a valid 'OpenSesame' experiment and a self-contained browser-based 'jsPsych' experiment, and the R and Python engines select identical materials in every case — including, for lexical decision, the very same deterministically generated pseudowords.

4 Discussion

'lexsync' unifies steps that are usually performed in separate tools and separate languages. Its contribution is integration rather than a single new capability: multidimensional matching exists in 'LexOPS' and 'LIBRA', and precise presentation exists in 'PsychoPy' and 'OpenSesame', but the path from a multilingual corpus to a hardware-timed, cross-platform experiment has not before been available as a single installable, dual-language tool. Two features are worth emphasising. First, the deterministic matcher selects identical matched (word, condition) sets in the R and Python engines, with the reported match statistics agreeing to the precision shown; only the seeded trial order, which depends on each language's random-number generator, differs. A laboratory can therefore adopt 'lexsync' in whichever ecosystem it prefers without fear of divergent stimulus selection. Second, both presentation targets upgrade the sequence-ordered triggers of the original workflow to onset-aligned markers — flip-locked in 'PsychoPy', and written immediately after the refresh-blocking `show()` in 'OpenSesame' — a concrete methodological improvement for ERP research.

Some limitations remain. The Chinese demonstration shows that the pipeline generalises to a logographic script — corpus access, dimension derivation, matching, counterbalancing and CJK-font script generation all work unchanged — but the neighbourhood measures it equates there (Coltheart's N and OLD20) are computed at the character level. They do not capture the sub-character structure that also shapes Chinese reading, such as radical, stroke-count and phonetic-radical information. Such measures are not intrinsic to the method; they enter as additional script-specific dimensions through the same extensible dimension system, and supplying them is left to future work. A photodiode check remains advisable for any onset-critical study, as for any software trigger. The example item tables for the paired and sentence paradigms are deliberately small, intended to exercise and document the pipeline rather than to constitute validated materials; researchers supply their own.

Several further extensions follow directly from the motivation and slot into the existing module structure. The web target above could be joined by a 'PCIbex' reading study and by client-side hardware synchronisation (WebSerial or a photodiode patch), so that online studies inherit the same onset markers the laboratory targets write. Richer dimensions — phonological (syllable count, phonotactic probability), semantic (concreteness, age of acquisition, valence) and predictability (n-gram or language-model surprisal) — would broaden the designs the matcher supports. Finally, resampling multiple matched item sets, or sampling items per participant, would let a study treat its items as the random factor they are (Clark, 1973; Yarkoni, 2020). A graphical interface is a further possibility.

5 Availability and open practices

'lexsync' is available as an R package and a Python package, with a shared corpus registry and the reproducible source of this manuscript. The code is released under the MIT licence and the bundled corpus derivatives under CC BY-SA 4.0, with full attribution and retrieval dates recorded alongside the data. The repository URL in the package metadata is a placeholder pending public release.

5.1 Extending lexsync

In keeping with the design, extensions have well-defined insertion points. To add a **paradigm**, add an entry to the registry in `paradigms` giving its default event sequence, the trial fields it requires and its counterbalancing recipe; both backends then render it with no further code. To add a **lexical dimension**, edit `compute`-style functions in `querying` (`add_neighbourhood` and the dimension table in `config/schema.yaml`) in both packages. To add a **presentation target**, add an `export_<target>()` function in `scripting` that walks the same rendered event list. To add a **corpus or language**, add an entry to `corpora/registry.yaml`; no code change is required for SUBTLEX-family or 'wordfreq' sources.

6 References

Bochynska, A., Keeble, L., Halfacre, C., Casillas, J. V., Champagne, I.-A., Chen, K., Röthlisberger, M., Buchanan, E. M., & Roettger, T. B. (2023). Reproducible research practices and transparency across linguistics. *Glossa Psycholinguistics*, *2*(1). https://doi.org/10.5070/G6011239

Brysbaert, M., & New, B. (2009). Moving beyond Kučera and Francis: A critical evaluation of current word frequency norms and the introduction of a new and improved word frequency measure for American English. *Behavior Research Methods*, *41*(4), 977–990. https://doi.org/10.3758/BRM.41.4.977

Cai, Q., & Brysbaert, M. (2010). SUBTLEX-CH: Chinese word and character frequencies based on film subtitles. *PLoS ONE*, *5*(6), e10729. https://doi.org/10.1371/journal.pone.0010729

Clark, H. H. (1973). The language-as-fixed-effect fallacy: A critique of language statistics in psychological research. *Journal of Verbal Learning and Verbal Behavior*, *12*(4), 335–359. https://doi.org/10.1016/S0022-5371(73)80014-3

Coltheart, M., Davelaar, E., Jonasson, J. T., & Besner, D. (1977). Access to the internal lexicon. In S. Dornic (Ed.), *Attention and performance VI* (pp. 535–555). Erlbaum.

Coretta, S., Casillas, J. V., Roessig, S., et al. (2023). Multidimensional signals and analytic flexibility: Estimating degrees of freedom in human-speech analyses. *Advances in Methods and Practices in Psychological Science*, *6*(3). https://doi.org/10.1177/25152459231162567

Cuetos, F., González-Nosti, M., Barbón, A., & Brysbaert, M. (2011). SUBTLEX-ESP: Spanish word frequencies based on film subtitles. *Psicológica*, *32*(2), 133–143.

González Alonso, J., Bernabeu, P., Silva, G., DeLuca, V., Poch, C., Ivanova, I., & Rothman, J. (2025). Starting from the very beginning: Unraveling third language (L3) development with longitudinal data from artificial language learning and EEG. *International Journal of Multilingualism*, *22*(1), 119–142. https://doi.org/10.1080/14790718.2024.2415993

Heuven, W. J. B. van, Mandera, P., Keuleers, E., & Brysbaert, M. (2014). SUBTLEX-UK: A new and improved word frequency database for British English. *Quarterly Journal of Experimental Psychology*, *67*(6), 1176–1190. https://doi.org/10.1080/17470218.2013.850521

Itkonen, S., Häikiö, T., Vainio, S., & Lehtonen, M. (2024). LASTU: A psycholinguistic search tool for Finnish lexical stimuli. *Behavior Research Methods*, *56*(6), 6165–6178. https://doi.org/10.3758/s13428-024-02347-x

Keuleers, E., & Brysbaert, M. (2010). Wuggy: A multilingual pseudoword generator. *Behavior Research Methods*, *42*(3), 627–633. https://doi.org/10.3758/BRM.42.3.627

Lakens, D. (2017). Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science*, *8*(4), 355–362. https://doi.org/10.1177/1948550617697177

Lintz, E. N., Lim, P. C., & Johnson, M. R. (2021). A new tool for equating lexical stimuli across experimental conditions. *MethodsX*, *8*, 101545. https://doi.org/10.1016/j.mex.2021.101545

Mathôt, S., Schreij, D., & Theeuwes, J. (2012). OpenSesame: An open-source, graphical experiment builder for the social sciences. *Behavior Research Methods*, *44*(2), 314–324. https://doi.org/10.3758/s13428-011-0168-7

Peirce, J., Gray, J. R., Simpson, S., MacAskill, M., Höchenberger, R., Sogo, H., Kastman, E., & Lindeløv, J. K. (2019). PsychoPy2: Experiments in behavior made easy. *Behavior Research Methods*, *51*(1), 195–203. https://doi.org/10.3758/s13428-018-01193-y

Roettger, T. B. (2019). Researcher degrees of freedom in phonetic research. *Laboratory Phonology*, *10*(1). https://doi.org/10.5334/labphon.147

Roettger, T. B., Winter, B., & Baayen, H. (2019). Emergent data analysis in phonetic sciences: Towards pluralism and reproducibility. *Journal of Phonetics*, *73*, 1–7. https://doi.org/10.1016/j.wocn.2018.12.001

Sassenhagen, J., & Alday, P. M. (2016). A common misapplication of statistical inference:

Nuisance control with null-hypothesis significance tests. *Brain and Language*, *162*, 42–45. https://doi.org/10.1016/j.bandl.2016.08.001

Speer, R. (2022). *rspeer/wordfreq: v3.0*. Zenodo. https://doi.org/10.5281/zenodo.7199437

Taylor, J. E., Beith, A., & Sereno, S. C. (2020). LexOPS: An R package and user interface for the controlled generation of word stimuli. *Behavior Research Methods*, *52*(6), 2372–2382. https://doi.org/10.3758/s13428-020-01389-1

Wilkinson, M. D., Dumontier, M., Aalbersberg, Ij. J., et al. (2016). The FAIR guiding principles for scientific data management and stewardship. *Scientific Data*, *3*, 160018. https://doi.org/10.1038/sdata.2016.18

Yarkoni, T. (2020). The generalizability crisis. *Behavioral and Brain Sciences*, *45*, e1. https://doi.org/10.1017/S0140525X20001685

Yarkoni, T., Balota, D., & Yap, M. (2008). Moving beyond Coltheart's N: A new measure of orthographic similarity. *Psychonomic Bulletin & Review*, *15*(5), 971–979. https://doi.org/10.3758/pbr.15.5.971